



Bachelors Degree Project

Impact of LSP Diagnostics on Agentic Code Translation Repair – *A Controlled Comparison of Raw Compiler Output and Structured LSP Feedback*



Authors: Ebbe Karlstad, Younus Mashoor

Supervisors: Morgan Ericsson (LNU), Tibo Bruneel (Softwerk)

Semester: VT 2026

Course: 2DV50E

Subject: Computer Science

Abstract

The abstract will be written when the experiment and analysis have been completed. It will summarize the background, research questions, methodology, and findings of the study.

Keywords: code translation, large language models, language server protocol, automated program repair, agentic pipelines

Preface

This thesis is written as part of the Bachelor's programme in Computer Science with a specialization in Software Technology at Linnaeus University, in cooperation with Softwerk AB.

We would like to thank everyone who helped us at Softwerk, and give a special thank you to Tibo Bruneel and Ingmar Falk, for their time, guidance, and for giving us an interesting problem to work on. Their knowledge and willingness to meet with us throughout the project has been a great help.

We also want to thank our academic supervisor Morgan Ericsson at Linnaeus University for his support and feedback along the way, and Daniel Toll for running the thesis course and for always being available when we had questions.

Contents

1	Introduction	1
1.1	Background	1
1.2	Related work	1
1.3	Problem formulation	2
1.4	Motivation	2
1.5	Results	3
1.6	Scope/Limitation	3
1.7	Target group	3
1.8	Outline	3
2	Method	4
2.1	Research Project	4
2.2	Research Methods	4
2.3	Reliability and Validity	5
2.4	Ethical Considerations	5
3	Theoretical Background	6
4	Research Project – Implementation	7
5	Results	8
6	Analysis	9
7	Discussion	10
8	Conclusions and Future Work	11
	References	12
A	Appendix 1	A

1 Introduction

Automated code translation is a critical workflow when it comes to modernizing legacy systems; however, Large Language Models (LLMs) often struggle with so-called "one-shot" translations, with correct translation rates ranging from only 2.1% to 47.3% [1], making an iterative repair loop, a process where the system repeatedly attempts to fix compilation errors using automated feedback, important to a translation pipeline.

Current pipelines [2, 3, 4] use raw compiler output as the feedback, which is often too ambiguous for an LLM to act upon. Little existing work investigates how the feedback quality matters for the LLMs, other than whether a repair loop helps at all.

To address this, we build an agentic pipeline, an automated system where an LLM iteratively evaluates and repairs its own output, that translates a codebase from one language to another (C++ $\rightarrow$ Rust), under two controlled conditions: one receiving only raw compiler output, and one that also receives structured diagnostics from the Language Server Protocol (LSP).

While raw compiler output provides unstructured error messages, LSP diagnostics are structured JSON objects which contain useful information like error codes, error severity levels, affected character ranges, and type hints.

By keeping all other variables in the pipeline constant, we isolate LSP access as the independent variable and measure its impact on translation and repair performance using compile success rates and the number of repair iterations required.

1.1 Background

Older critical systems still use languages like COBOL, Fortran, and Delphi, and with developers moving away to more modern languages, these systems become harder and more expensive to maintain. In situations where a rewrite of these legacy systems is needed, doing so manually is often not viable. Rewriting by hand is simply too expensive and slow. A Danfoss Power Solutions project estimated that a manual rewrite from Delphi to C# would take 3,700 person-months to complete [2].

LLMs make automated translation of this scale more realistic than manual rewriting, but often fail at the compilation step. Research shows that compilation failures specifically account for roughly 77.8% of unsuccessful translations [1], making a post-translation repair loop a necessary component in order to ensure the success of these translations.

1.2 Related work

Academically, this thesis sits between two different research areas: automated program repair and code translation. LSP has been shown to help in single-language code repair [5], and Go et al. [6] applied LSP diagnostics to iterative test generation across multiple different languages. However, neither addresses cross-language code repair. Existing pipelines like TRANSAGENT [3], UniTrans [4], and the pipeline built at Softwerk [2] all confirm that a debugging stage is necessary, but no research investigates the feedback signal that's driving it.

Bruneel et al. [2] built an engineering-driven pipeline to translate a large codebase from Delphi to C#, and confirmed that an iterative repair loop is necessary for projects of this size. Their repair loop relies only on raw compiler output, which leaves the quality of the feedback signal untested. We build directly on this pipeline architecture and extend it by introducing structured LSP diagnostics as an alternative feedback signal.

Wei et al. [5] built Repilot, and used LSP diagnostics to guide and constrain LLM-based repair in Java. They concluded that this method raised compilation rates from 43% to 63%, and is the closest existing work to our thesis proposal, but it operates at generation time for a single language.

Go et al. [6] used LSP diagnostics in an iterative repair loop for unit test generation across multiple languages (Java, Go, Python). This uses multiple languages in the post-generation phase, but targets test generation, not code translation repair. Yang et al. [4] did research on code translation by building a framework (UniTrans) that uses test execution to iteratively repair translated code. They found that this approach led to substantial improvements across six language pairs. TRANSAGENT [3] builds directly on top of this, and outperformed UniTrans by up to 33.3%.

The difference between UniTrans and TRANSAGENT and our approach is that we isolate the feedback signal as an independent variable which the two mentioned papers do not do. Pan et al. [1] studied translation bugs across multiple LLM models and language pairs, concluding that the majority of failures are compilation errors. Ouyang et al. [7] studied LLM non-determinism in code generation, the tendency of models to produce different outputs for the same prompts, and concluded that these outputs vary even at temperature=0 (which often is assumed to produce deterministic output), which motivates our use of multiple pipeline runs.

1.3 Problem formulation

Few studies have properly studied what kind of feedback works the best in this repair loop, and there is little to no current research where the feedback signal is controlled as an independent variable. Furthermore, no studies have compared raw compiler output against structured LSP diagnostics in code translation repair between two languages. As established in the background and related work, the repair loop is a necessary component in LLM-based translation pipelines, yet the quality of the feedback signal driving it remains unexamined. To address this gap, we apply LSP diagnostics to the repair loop within our code translation pipeline.

- RQ1.** To what extent does LSP-augmented feedback improve the compile/type-check success rate of an agentic repair loop compared to raw compiler output, after a certain number of repair iterations?
- RQ2.** How does LSP access affect the number of repair iterations required to reach the first successful compilation?
- RQ3.** Which categories of errors does the LSP-augmented repair loop resolve that the baseline does not?

1.4 Motivation

The feedback signal in code repair loops has not been isolated as a controlled variable in prior work, and this study produces the first controlled test, comparing raw compiler output against LSP diagnostics in cross-language translation code repair. Results will directly affect how future similar repair loops are designed.

Large-scale legacy code translation projects, similar to the Danfoss migration, are expensive and time consuming [2]. We believe our research can improve the design of these migration projects. By researching the effectiveness of LSP access in automated

code translation repair loops, we aim to provide guidance to the teams migrating these large projects on which feedback signal to use.

Critical systems in finance and healthcare often run on old legacy code, and keeping these systems maintained by migrating them to modern languages is essential to their integrity and performance. Better automated code translation tools reduce the barrier to modernizing these legacy systems in a safe and efficient way.

1.5 Results

We expect the produced results of our experiment to be compilation success rates per condition per component after N repair iterations. According to our proposed design, we also expect to produce the number of iterations to reach the first successful compilation. Additionally, we plan to categorize the error types for each condition, and report them, to address RQ3.

1.6 Scope/Limitation

We aim to build a code translation pipeline that works for the languages C++ $\rightarrow$ Rust specifically. We compare exactly two feedback conditions, raw `stderr` against structured LSP diagnostics. We measure compilation success rate and iterations to first success. We use two open source datasets; TinyXML2 and SQLiteCpp, and we test with one specific LLM model.

We do not test semantic correctness, more than one language pair, or more than one LLM model, and the datasets do not reflect a full production environment.

1.7 Target group

This thesis is targeted at researchers within the field of automated program repair and LLM-based code translation who want evidence on the effect the feedback signal quality has on the translation effectiveness and correctness.

It is also targeted towards engineers building translation pipelines, and companies and teams migrating large-scale legacy code from one language to another, especially the chosen language pair C++ $\rightarrow$ Rust.

1.8 Outline

Chapter 2 covers the research methodology used, explains the general design, dataset selection, and validity concerns. Chapter 3 explains LLM-based code translation, automated program repair, and the Language Server Protocol. Chapter 4 covers the actual implementation of the code translation pipeline, the repair agent, and any other code that might be interesting to the reader. Chapter 5 demonstrates the raw results and findings, and Chapter 6 analyses the results with respect to the research questions. Chapter 7 discusses the findings and what they mean for the research field. Finally, Chapter 8 concludes the thesis itself, and covers details about future work.

2 Method

The research questions for this thesis are comparative and quantitative, and our main goal is to measure whether one feedback signal outperforms the other based on certain metrics. The experiment we run is controlled, with only a single independent variable, that being the type of feedback signal provided. All other factors remain constant. Prior to the experiment, we also considered other alternatives which we dismissed. Surveys and interviews, which collect data from human participants about their experiences, were considered unsuitable as we are measuring system behavior, not human perception. A case study, which investigates a phenomenon in its real-world context without controlling any variables, would introduce too many uncontrolled variables. A systematic literature review, which synthesizes existing published research, does not produce new empirical data. Because of this, we deem the experiment we run to be the best option available.

2.1 Research Project

This project is conducted in cooperation with Softwerk AB, who provided access to domain expertise and whose prior work on LLM-based code translation pipelines informed our own design.

We began by conducting a literature review in order to confirm the research gap. Following this, we selected the language pair (C++ $\rightarrow$ Rust) to run our experiment on and the appropriate datasets (TinyXML2 and SQLiteCpp), which are both open sourced under the MIT license and have a reasonable size considering our time constraints. Finally, we configured the tool chains which include the agent framework (Agno), the compiler, the LSP server (rust-analyzer) and the logging infrastructure.

With the foundation in place, we are implementing the translation pipeline in Python using the Agno framework. The pipeline runs under two conditions; Condition A (repair agent receives raw compiler `stderr`) and Condition B (repair agent receives LSP diagnostics). The translation stage is identical across both conditions. To prevent an infinite loop and unnecessary API costs we enforce a fixed iteration cap per component. At each step we log the state of the code, any feedback signals, and the compilation results.

2.2 Research Methods

Each code component is translated and repaired under both conditions, making this a paired design. Condition A feeds the repair agent raw compiler `stderr`. Condition B feeds it structured LSP diagnostics, which include severity levels, diagnostic codes, character ranges, and type hints. Because the same components appear in both conditions, differences in component complexity do not affect the comparison.

The independent variable is the feedback signal type. The primary dependent variable is the compilation success rate after N repair iterations. Secondary metrics include the number of iterations needed to reach the first successful compilation and the categories of errors resolved. The LLM model, prompt template, translation stage, iteration cap, and dataset are held constant across both conditions.

Because of LLMs non-deterministic nature [7], we run each condition multiple times per component and aggregate the results before we compare them. The outcome for each component under each condition is binary; the code either compiles or it does not.

Because the data is paired, we use McNemar’s test to compare success rates across conditions. For secondary metrics we report the median iterations to the first compilation success and the distribution of error categories that were resolved.

We select self-contained, unit-level components from our two proposed open source repositories. Components must be complex enough to produce non-trivial type and compilation errors while remaining within our LOC constraint. Before committing to the full dataset, we run preliminary tests on a small subset of code components to confirm that the components produce a meaningful range of errors.

We conducted a focused literature review using targeted searches across relevant venues to confirm that no prior work isolates the feedback signal as a controlled variable in cross-language translation repair. The domain is narrow enough that a full systematic literature review was not necessary.

2.3 Reliability and Validity

Several threats to validity have been identified in our experimental design. LLM non-determinism is a key internal threat, as identical prompts can produce different outputs across runs. We address this by running each condition multiple times per component and aggregating results. Prompt design is another concern, since Condition B inherently receives more verbose feedback than Condition A, meaning the prompt length rather than the diagnostic content could influence the outcome. We mitigate this by keeping the prompt template structure identical across conditions. Results are bound to our chosen language pair and LLM model, which limits generalizability.

Regarding likelihood, LLM non-determinism represents a high-likelihood threat, while prompt verbosity is a medium-likelihood concern. A more detailed discussion of reliability and validity threats will be provided once the experiment has been conducted and all the threats can be assessed.

2.4 Ethical Considerations

This work involves no personal data and no human participants, making GDPR considerations non-applicable. All datasets consist of open-source code used in compliance with their respective licenses. Results, including negative results, will be reported in full to avoid publication bias. No risks of harm to individuals or society have been identified in this work.

3 Theoretical Background

4 Research Project – Implementation

5 Results

6 Analysis

7 Discussion

8 Conclusions and Future Work

References

- [1] R. Pan, M. C. Csallner, C. Wang, and Y. Liu, “Lost in translation: A study of bugs introduced by large language models,” in *Proc. 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.
- [2] T. Bruneel, W. Löwe, M. Ericsson, D. Perez-Palacin, and J. Nordqvist, “Automated code translation using an engineering-driven pipeline approach of language models,” in *Proc. Workshop on Machine Learning Operations (MLOps25)*, ser. CEUR Workshop Proceedings, vol. 4109, 2025. [Online]. Available: <https://ceur-ws.org/Vol-4109/paper5.pdf>
- [3] Z. Yuan, W. Chen, H. Wang, K. Yu, X. Peng, and Y. Lou, “TRANSAGENT: An LLM-based multi-agent system for code translation,” in *Proc. ACM International Conference on the Foundations of Software Engineering (FSE’26)*, 2026. [Online]. Available: <https://arxiv.org/abs/2409.19894>
- [4] Z. Yang, F. Liu, Z. Yu, J. W. Keung, J. Li, S. Liu, Y. Hong, X. Ma, Z. Jin, and G. Li, “Exploring and unleashing the power of large language models in automated code translation,” in *Proc. ACM International Conference on the Foundations of Software Engineering (FSE’24)*, 2024.
- [5] Y. Wei, C. S. Xia, and L. Zhang, “Copiloting the copilots: Fusing large language models with completion engines for automated program repair,” in *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE ’23)*, San Francisco, CA, USA, 2023, pp. 172–184.
- [6] G. Go, Q. Zhang, C. Zhou, Z. Wei, and Y. Jiang, “LSPRAG: LSP-guided RAG for language-agnostic real-time unit test generation,” in *Proc. ACM International Conference on the Foundations of Software Engineering (FSE’26)*, 2026.
- [7] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, “An empirical study of the non-determinism of ChatGPT in code generation,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 2, 2025.

A Appendix 1